

NAIVE BAYES



Table of contents

01

About Naive Bayes

02

Types of Naive Bayes

03

Pros of Naive Bayes

04

Cons of Naive Bayes

05

Bayes Theorem

06

Understanding Naive Bayes

07

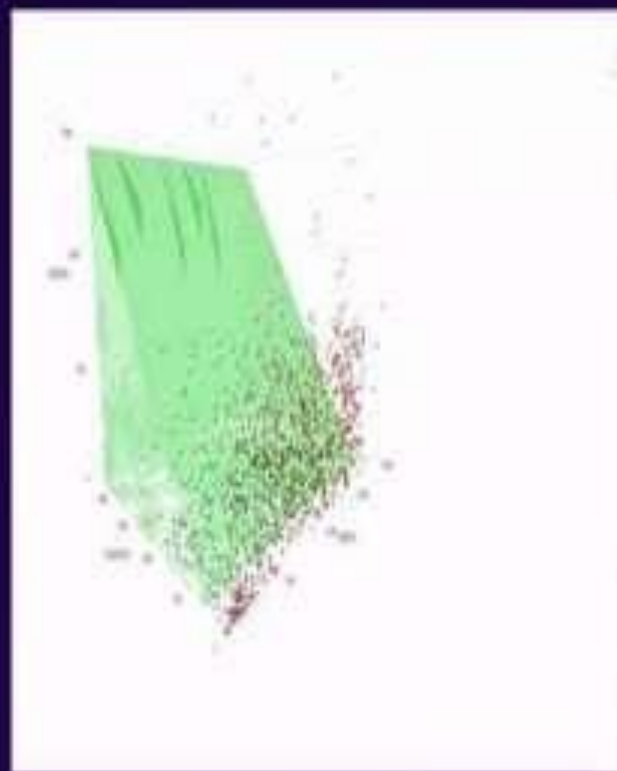
When to use Naive Bayes

08

Application of Naive Bayes

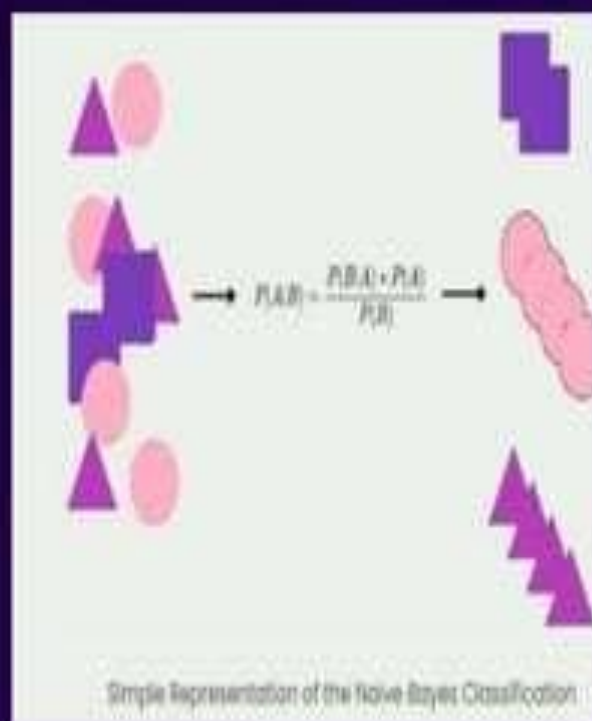
What Is Naive Bayes?

- Naive Bayes is a supervised learning algorithm, based on Bayes theorem and used to solve classification problems.
- It can be easily written in code and predictions can be made real quick, which in turn increases the scalability of the solution.
- It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.



What Is Naive Bayes?(Cont.)

- Naive Bayes Classifier is one of the simple and most effective classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- Some popular examples of Naive Bayes Algorithm are **spam filtration**, **Sentimental analysis**, and **classifying articles**.



Why is it called Naive Bayes?

Naive Bayes algorithm is comprised of two words **Naive** and **Bayes**, which can be Summarized as:

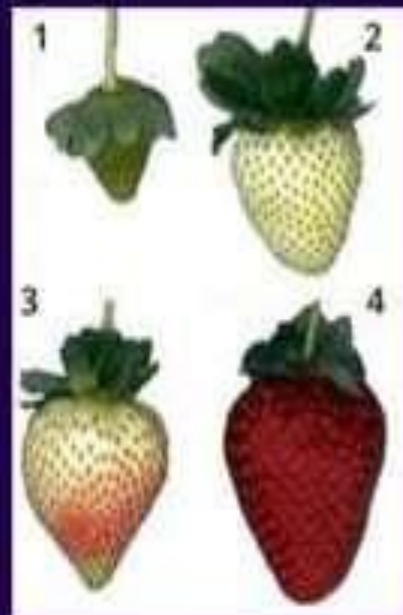
Naive — It is called Naive because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Which might not be true in real life.

Example: If a strawberry is ripe or not to harvest?

Assume, two independent features in this case: **Size** and **Color**. To demonstrate the assumptions are partially invalid, pose the following two critical questions:

Why is it called Naive Bayes?(Cont.)

- Size and Color are independent? Not really, a positive correlation could be seen apparently. The fruit grows, expanding in size and changing in color.
- Size and Color equally contribute to the outcome of "Being ripe"? Not really! Although it greatly depends on the type of fruit, we can roughly anticipate the answer.



Bayes — It is called Bayes because it depends on the principle of Bayes' Theorem.

How do Naive Bayes Algorithm work?

- Convert the data set into a frequency table.
- Create Likelihood table by finding the probabilities.
- Use Naive Bayesian equation to calculate the posterior probability.

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Types of Naive Bayes Classifier

- **Gaussian Naive Bayes** — In a Gaussian Naive Bayes, the predictors take a continuous value assuming that it has been sampled from a Gaussian Distribution. It is also called a Normal Distribution.
- **Multinomial Naive Bayes** — These types of classifiers are usually used for the problems of document classification. It checks whether the document belongs to a particular category like sports or technology or political etc and then classifies them accordingly. The predictors used for classification in this technique are the frequency of words present in the document.
- **Bernoulli Naive Bayes** — This classifier is also analogous to multinomial naive bayes but instead of words, the predictors are Boolean values. The parameters used to predict the class variable accepts only yes or no values, for example, if a word occurs in the text or not.

Pros & Cons of Naive Bayes

Pros of Naive Bayes —

- It is **easy** and **fast** to predict the class of the training data set.
- It performs well in **multiclass** prediction.
- It performs better as compared to other models like **logistic regression** while assuming the independent variables.
- It requires less training data.
- It performs better in the case of **categorical** input variables as compared to **numerical** variables.

Pros & Cons of Naive Bayes(Cont.)

Cons of Naive Bayes-

- The model will assign a 0 (zero) probability and will be unable to make a prediction if categorical variable has a category (in test data set). This is often known as "Zero Frequency". To solve this, we can use the smoothing technique like "Laplace estimation".
- Another limitation of this algorithm is the assumption of independent predictors. In real life, it is almost impossible that we get a set of predictors which are completely independent.
- Naive Bayes is also known as a **bad estimator**, so the probability outputs are not taken seriously.

Bayes Theorem

Bayes' theorem helps you examine the probability of an event based on the prior knowledge of any event that has correspondence to the former event. Its uses are mainly found in probability theory and statistics. The formula for Bayes' theorem is given as:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Where, A and B are called events.

- $P(A | B)$ is the probability of event A, given the event B is true (has occurred). Event B is also termed as evidence.
- $P(A)$ is the priori of A (the prior independent probability, i.e. probability of event before evidence is seen).
- $P(B)$ is the priori of B (the prior independent probability, i.e. probability of event before evidence is seen).
- $P(B | A)$ is the probability of B given event A, i.e. probability of event B after evidence A is seen.

Naive Bayes Example (Buying a Computer)

Problem:

You are given some past **customer data** about **Age**, **Income**, and whether they **bought a computer** (Yes or No).

Now, a **new customer** is **Young** and has **Medium** income.

You need to predict:

Will they buy a computer?

Problem: Predict if a person will buy a computer based on Age and Income.

Person	Age	Income	Buys Computer
1	Young	High	No
2	Young	High	No
3	Middle	High	Yes
4	Senior	Medium	Yes
5	Senior	Low	Yes
6	Senior	Low	No
7	Middle	Low	Yes
8	Young	Medium	No
9	Young	Low	Yes
10	Senior	Medium	Yes
11	Young	Medium	Yes
12	Middle	Medium	Yes
13	Middle	High	Yes
14	Senior	Medium	No

Step-by-Step Breakdown

1. Prior Probabilities

First, we need to know **how likely overall** customers buy or not buy.

From the table:

- 9 customers said **Yes** (they bought a computer).
- 5 customers said **No**.

So:

$$P(\text{Yes}) = \frac{9}{14} \approx 0.643$$

$$P(\text{No}) = \frac{5}{14} \approx 0.357$$

This means that, **without any extra information**, buying is more common.

2. Likelihood Probabilities

Now we use the new customer's features (**Young, Medium**) to update our prediction.

We calculate:

- How often **Young** people bought / did not buy.
- How often **Medium Income** people bought / did not buy.

For Age = Young

- Among customers who **bought** (Yes), 2 of them were **Young**.
- Among customers who **did not buy** (No), 3 were **Young**.

Calculations:

$$P(\text{Young}|\text{Yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{Young}|\text{No}) = \frac{3}{5} = 0.6$$

- So if you **bought**, you were Young only about **22%** of the time.
- If you **didn't buy**, you were Young **60%** of the time.

For Income = Medium

- Among customers who **bought** (Yes), 4 had **Medium** income.
- Among customers who **did not buy** (No), 1 had **Medium** income.

Calculations:

$$P(\text{Medium}|\text{Yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{Medium}|\text{Yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{Medium}|\text{No}) = \frac{1}{5} = 0.2$$

- So about **44%** of buyers had Medium income.
- Only **20%** of non-buyers had Medium income.

3. Calculate the Total Score

Use Bayes Theorem (Naive because we assume independence):

$$\begin{aligned}P(\text{Yes}|\text{Young, Medium}) &\propto P(\text{Yes}) \times P(\text{Young}|\text{Yes}) \times P(\text{Medium}|\text{Yes}) \\&= 0.643 \times 0.222 \times 0.444 \\&= 0.0635\end{aligned}$$

Similarly:

$$\begin{aligned}P(\text{No}|\text{Young, Medium}) &\propto P(\text{No}) \times P(\text{Young}|\text{No}) \times P(\text{Medium}|\text{No}) \\&= 0.357 \times 0.6 \times 0.2 \\&= 0.0428\end{aligned}$$

4. Compare

- Probability of buying: **0.0635**
- Probability of not buying: **0.0428**

✓ Since $0.0635 > 0.0428$,

✓ We **predict: YES, the customer will buy a computer!**

When to use Naive Bayes

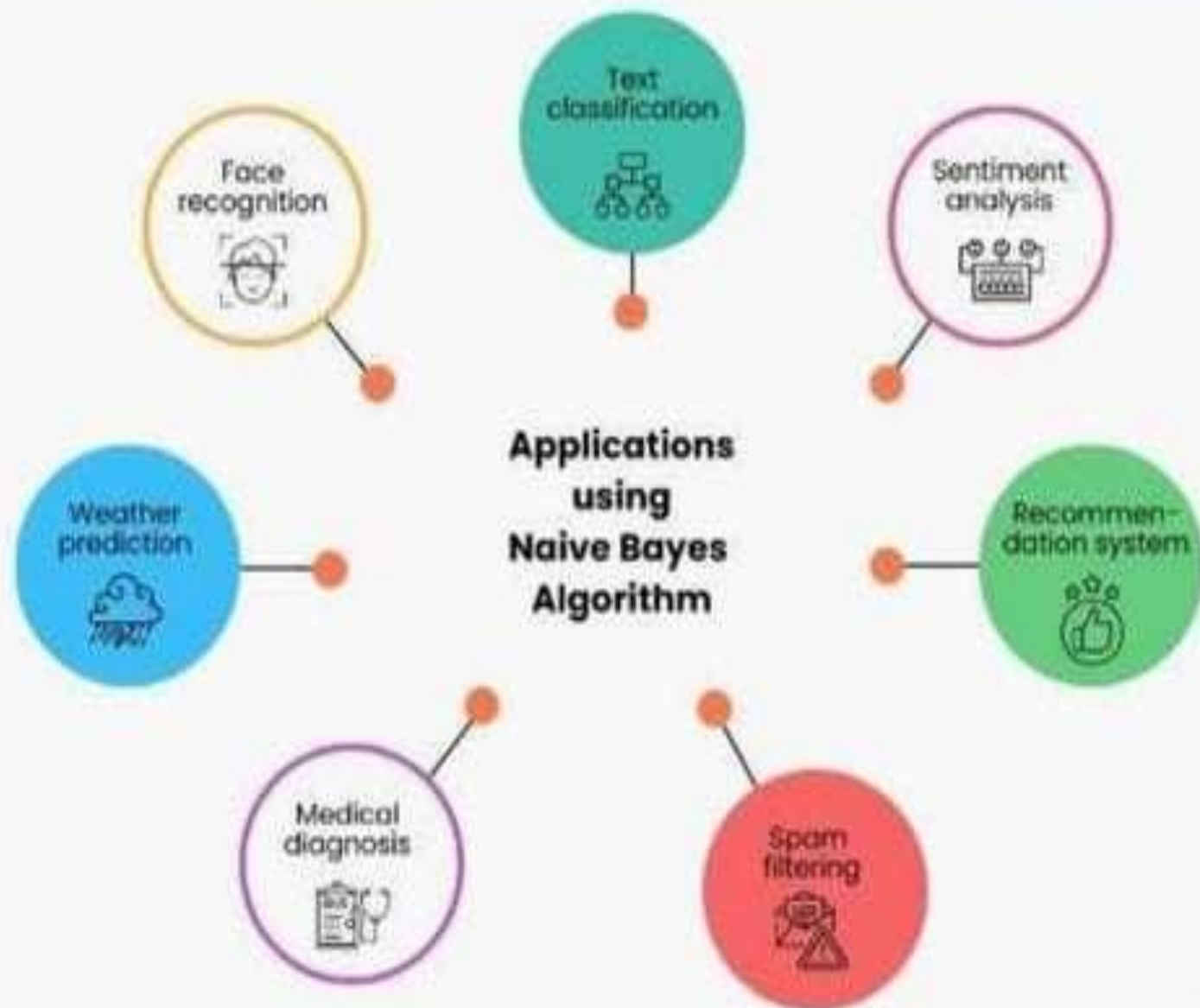
A Naive Bayesian classifier performs worse than a complex classifier due to the strict assumptions it makes about the data. The classifier, however, has some patterns:

- Training and predicting the model is done at a high speed.
- Probabilistic predictions can be created purely based on the data.
- They are usually pretty easy to interpret.
- Their parameters are usually not tunable.

When to Use Naive Bayes

- 1 High-speed training and prediction
- 2 Probability prediction based on data
- 3 Easy to interpret
- 4 Non-tunable parameters

Application of Naive Bayes Classifier



Application of Naive Bayes Classifier

In the field of machine learning, Naive Bayes is used as a classification model, i.e. to classify a data set into a certain class. There are various concrete applications for these models for which Naive Bayes is also used:

- **Real-time prediction.**
- **Multi-class prediction.**
- **Recommendation Systems.**
- **Classification of credit risk.**
- **Face Recognition.**
- **Predict Medical treatment.**
- **Text Classification/ Spam Filtering / Sentiment Analysis :**

They are mostly used in text classification problems because of its multi-class problems and the independence rule. They are used for identifying spam emails and also to identify negative and positive customer sentiments on social platforms.

Home Task

Naive Bayes Example: Spam Email Detection

Problem:

You are building a simple model to **predict if an email is Spam** based on the presence of certain words:

Email ID	"Free"	"Win"	"Money"	Spam
1	Yes	Yes	No	Yes
2	Yes	No	Yes	Yes
3	No	Yes	No	No
4	No	No	Yes	No
5	Yes	Yes	Yes	Yes
6	No	Yes	Yes	No

New Email to classify:

- Contains "**Free**" = **Yes**, "**Win**" = **Yes**, "**Money**" = **No**
- Predict if it's **Spam** or **Not Spam**!
- Python code is also required for the task with output.

**Thank
you!_**

